UNIVERSITY OF GHANA

College of Basic and Applied Sciences
School of Engineering Sciences

CPEN 208: Introduction to Software Engineering

Second Semester 2025/2026

PROJECT 2 REPORT

Name: Jude Addo
Index Number: 22385323
Programme: BSc. Computer Engineering

GitHub Repository: github.com/judeaddo23/cpen208-project2

1. Introduction

This report documents Project 2 for CPEN 208: Introduction to Software Engineering, which required building an API/web service that implements the functions created in Project 1. The result is a standalone Express.js REST API that connects directly to the same PostgreSQL database built in Project 1 (cpen_dept_db) and exposes each of its five required functionalities, plus the outstanding-fees calculation, as HTTP endpoints.

Project 2 was deliberately built as a separate application from Project 1's Next.js frontend, rather than folded into it. This mirrors the brief's framing of Project 1 and Project 2 as two distinct deliverables (each with its own repository and report) and demonstrates the API functioning independently of any particular frontend, the same way a real backend service would be consumed by multiple different clients.

2. Architecture and Design

- The API is built with Express.js on Node.js, using plain JavaScript.
- It connects to the same cpen_dept_db PostgreSQL database and academic schema built in Project 1, via the pg (node-postgres) package, no data or schema was duplicated.
- Routes are organised by resource into separate files under routes/ (fees.js, students.js, courses.js, lecturers.js), each exporting an Express Router, and mounted onto the main app in server.js under a matching URL prefix (/api/fees, /api/students, /api/courses, /api/lecturers).
- cors is enabled so the API can be called from a browser-based frontend on a different port (e.g. Project 1's Next.js app), and express.json() is used to parse JSON request bodies for POST endpoints.
- nodemon is used in development (npm run dev) to automatically restart the server on file changes.
- Database credentials are stored in a .env file (DATABASE_URL, PORT), excluded from version control via .gitignore.

3. API Endpoints

The table below lists every endpoint implemented, grouped by the Project 1 functionality each one supports.

Endpoint	Description
`GET /api/fees/outstanding`	Returns academic.get_outstanding_fees() as a JSON array (amount due, paid, and outstanding balance per student).
`POST /api/fees/payments`	Records a new fee payment for a student. Body: student_id, amount_paid, semester.
`GET /api/students`	Lists all students.
`GET /api/students/:id`	Returns one student by ID.
`POST /api/students`	Creates a new student. Body: index_number, first_name, last_name, email, program, level.
`GET /api/courses`	Lists the course catalogue.
`GET /api/courses/:code/students`	Lists everyone enrolled in a given course.
`POST /api/courses/enroll`	Enrolls a student in a course. Body: student_id, course_code, semester.
`GET /api/lecturers`	Lists all lecturers.
`GET /api/lecturers/:id/courses`	Lists the courses a lecturer teaches.
`POST /api/lecturers/assign-course`	Assigns a lecturer to a course. Body: lecturer_id, course_code, semester.
`GET /api/lecturers/:id/tas`	Lists the TAs assigned to a lecturer.
`POST /api/lecturers/assign-ta`	Assigns a TA to a lecturer for a course. Body: lecturer_id, ta_id, course_code, semester.

Together, these endpoints cover all five functionalities required in Project 1 (student personal information, student fee payments, course enrollment, lecturer-to-course assignment, and lecturer-to-TA assignment), plus the outstanding-fees JSON function.

4. Testing

GET endpoints were tested directly in the browser. For example, navigating to http://localhost:4000/api/fees/outstanding returns the same JSON array produced by academic.get_outstanding_fees() in Project 1, confirming the API correctly calls the underlying database function.

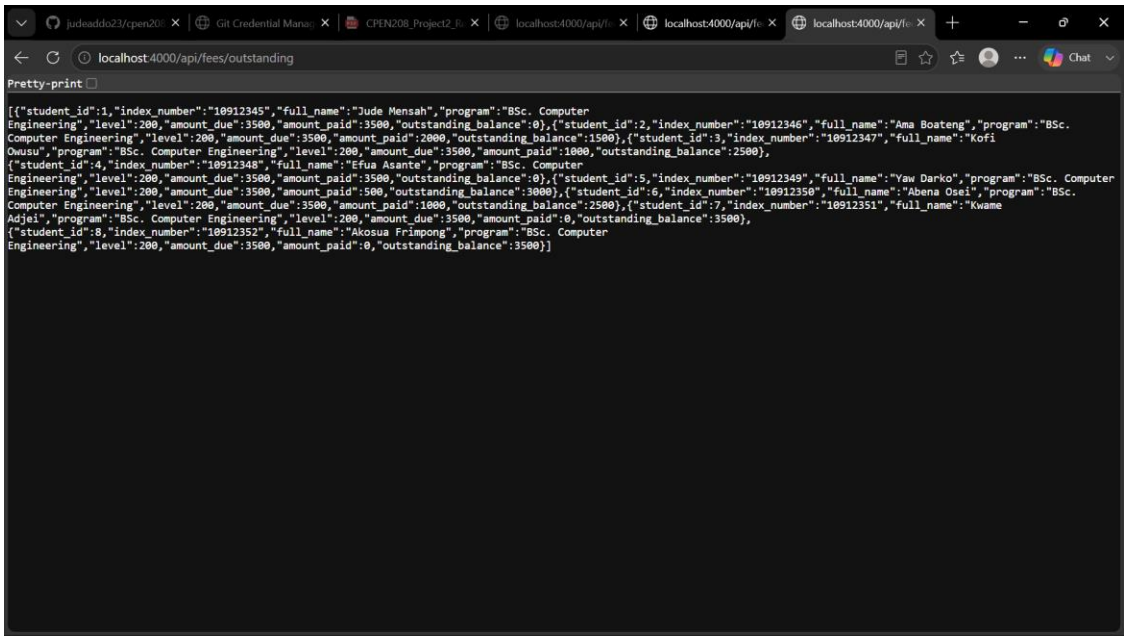

Figure 1: GET /api/fees/outstanding response

POST endpoints, which require a JSON request body, were tested using the Thunder Client extension for VS Code. A payment of GHS 1,000.00 was submitted for a student via POST /api/fees/payments, which returned a 201 Created response with the new payment record. The change was then verified two ways: directly in the database (a new row in academic.fee_payments), and by re-querying GET /api/fees/outstanding, which showed that student's outstanding_balance correctly reduced by GHS 1,000.00, confirming the full pipeline (API → database → outstanding-fees calculation) works end to end.

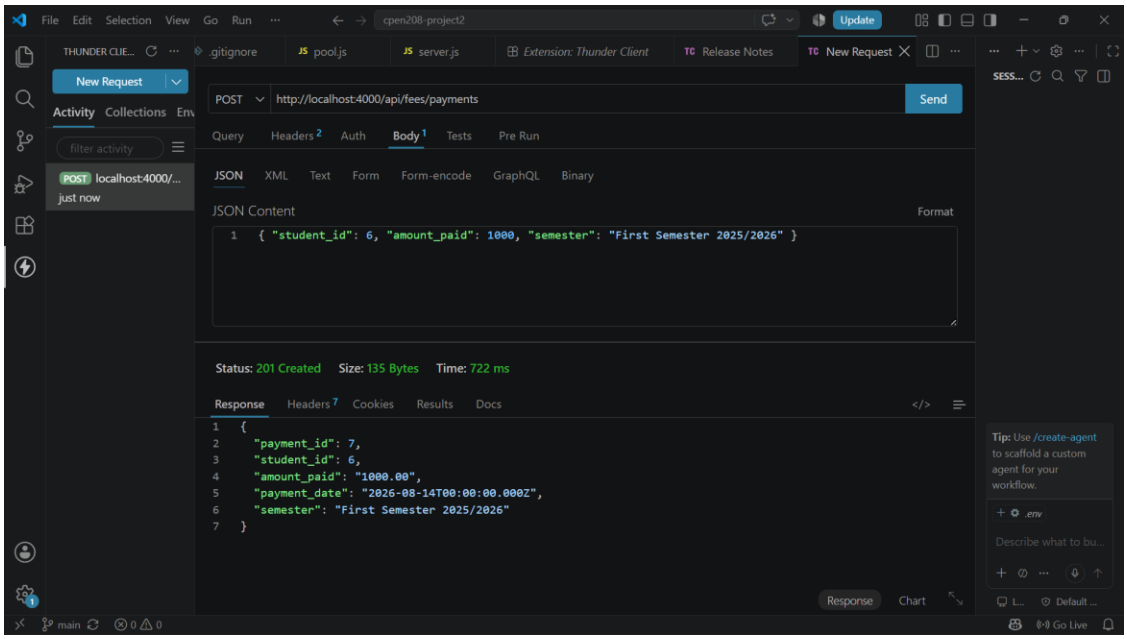

Figure 2: POST /api/fees/payments via Thunder Client

5. Database

Project 2 uses the same cpen_dept_db database, schema, tables, and sample data built in Project 1, no separate database or backup was created for Project 2, since it operates on the identical dataset. The database backup (cpen_dept_db_backup.sql) included with Project 1's submission applies to both projects.

6. GitHub Repository

All source code for the API has been pushed to GitHub:

https://github.com/judeaddo23/cpen208-project2

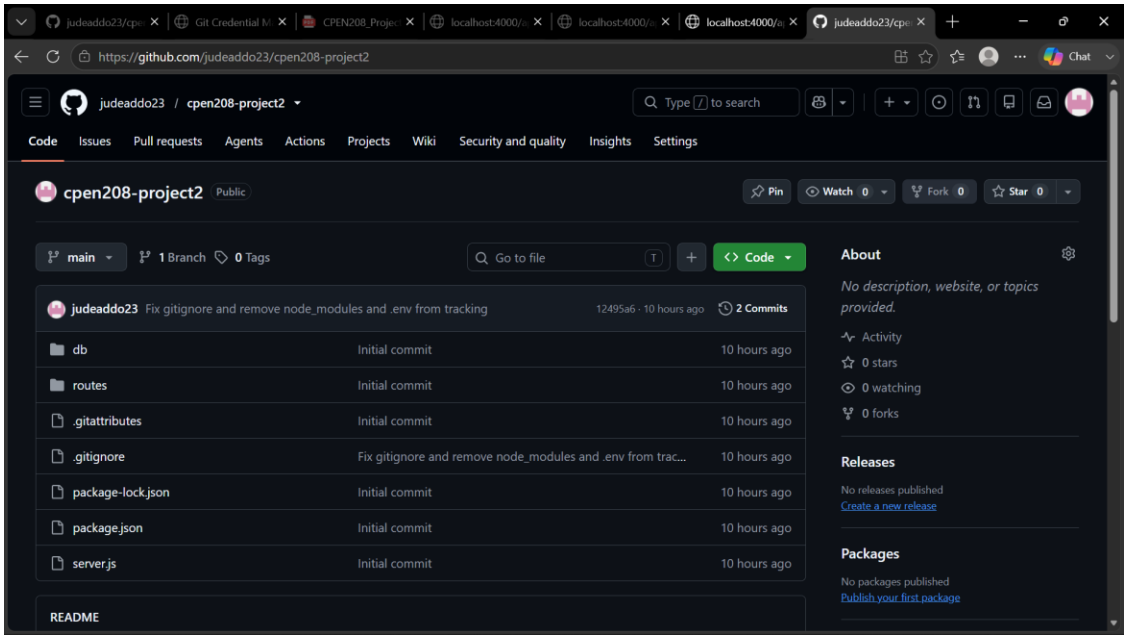

Figure 3: Project 2 GitHub repository

7. Conclusion

Project 2 successfully delivers a standalone Express.js REST API implementing all five functionalities from Project 1's database, plus the outstanding-fees calculation, tested end to end for both reads and writes. The API and Project 1's Next.js application both operate on the same underlying PostgreSQL database, demonstrating that the data layer built in Project 1 can be consumed by multiple independent applications.